

【数组拼接】

题目描述

现在有多组整数数组，需要将它们合并成一个新的数组。

合并规则，从每个数组里按顺序取出固定长度的内容合并到新的数组中，取完的内容会删除掉，如果该行不足固定长度或者已经为空，则直接取出剩余部分的内容放到新的数组中，继续下一行。

输入描述

第一行是每次读取的固定长度， $0 < \text{长度} < 10$

第二行是整数数组的数目， $0 < \text{数目} < 1000$

第3-n行是需要合并的数组，**不同的** 数组用回车换行分隔，数组内部用逗号分隔，最大不超过100个元素。

输出描述

输出一个新的数组，用逗号分隔。

用例

输入	3 2 2,5,6,7,9,5,7 1,7,4,3,4
输出	2,5,6,1,7,4,7,9,5,3,4,7
说明	1、获得长度3和数组数目2 2、先遍历第一行，获得2,5,6

	3、再遍历第二行，获得1,7,4 4、再循环回到第一行，获得7,9,5 5、再遍历第二行，获得3,4 6、再回到第一行，获得7，按顺序拼接成最终结果
--	---

输入	4 3 1,2,3,4,5,6 1,2,3 1,2,3,4
输出	1,2,3,4,1,2,3,1,2,3,4,5,6
说明	无

题目解析

假设，第一行输入的固定长度是subLen。

我的解题思路如下：

首先，定义一个 动态数组 ans，来记录题解。

然后，将输入的3~n行对应的数组，放到一个链表中。

每次取出链表的头部，然后删除该头部数组前subLen个元素，并这subLen元素记录按顺序录入ans中：

- 如果删除后，该头部数组长度依旧大于0，则将该头部数组重新连接到链表尾部
- 如果删除后，该头部数组的长度为0，则不做操作

直到链表长度为0时，停止。此时ans中记录的就是题解。

本题，主要难点在于，如何删除数组（或其他容器）的前subLen个元素，其实就是删除容器中某个范围的所有元素。在下面代码中，我针对不同语言，各附带了一个链接，里面有说明如何删除对应容器的某个范围的元素的策略。

根据考友反馈，本题实际考试时，会出现下面这种用例输入

```
2
2
1,2,3,4,5
4,,6,,7
```

因此输入解析时需要注意。

Java算法源码

```
1 import java.util.*;
2 import java.util.stream.Collectors;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         int subLen = Integer.parseInt(sc.nextLine());
9
10        int n = Integer.parseInt(sc.nextLine());
11
12        LinkedList<LinkedList<Integer>> lists = new LinkedList<>();
13        for (int i = 0; i < n; i++) {
14            lists.add(
15                new LinkedList<>()
16                    .stream(sc.nextLine().split(","))
17                    // 输入用例中的数组存在 4,,6 这种情况，所以这里需要过滤掉空串
18                    .filter(a -> !"".equals(a))
19                    .map(Integer::parseInt)
20                    .collect(Collectors.toList()));
21        }
22
23        System.out.println(getResult(subLen, lists));
```

```

24 | }
25 |
26 | public static String getResult(int subLen, LinkedList<LinkedList<Integer>> lists) {
27 |     ArrayList<Integer> ans = new ArrayList<>();
28 |
29 |     while (lists.size() > 0) {
30 |         LinkedList<Integer> list = lists.removeFirst();
31 |         ans.addAll(removeRange(list, 0, subLen));
32 |         if (list.size() > 0) lists.addLast(list);
33 |     }
34 |
35 |     StringJoiner sj = new StringJoiner(",");
36 |     for (Integer an : ans) {
37 |         sj.add(an + "");
38 |     }
39 |     return sj.toString();
40 | }
41 |
42 | public static List<Integer> removeRange(LinkedList<Integer> list, int start, int end) {
43 |     if (end > list.size()) end = list.size();
44 |
45 |     List<Integer> tmp = list.subList(start, end);
46 |
47 |     List<Integer> ans = new ArrayList<>(tmp);
48 |     tmp.clear();
49 |
50 |     return ans;
51 | }
52 | }

```

JS算法源码

```

1 | /* JavaScript Node ACM模式 控制台输入获取 */
2 | const readline = require("readline");
3 |

```

```
4 | const rl = readline.createInterface({ 5 |   input: process.stdin,
6 |     output: process.stdout,
7 |   });
8 |
9 | const lines = [];
10 | rl.on("line", (line) => {
11 |   lines.push(line);
12 |
13 |   if (lines[1] && lines.length === 2 + parseInt(lines[1])) {
14 |     const subLen = lines.shift() - 0;
15 |     lines.shift();
16 |     const lists = lines.map((line) =>
17 |       line
18 |         .split(",")
19 |         // 输入用例中的数组存在 4,,6 这种情况，所以需要过滤掉空串
20 |         .filter((a) => a !== "")
21 |         .map(Number)
22 |     );
23 |
24 |     console.log(getResult(subLen, lists));
25 |
26 |     lines.length = 0;
27 |   }
28 | });
29 |
30 | function getResult(subLen, lists) {
31 |   const ans = [];
32 |
33 |   while (lists.length > 0) {
34 |     const list = lists.shift();
35 |     ans.push(...list.splice(0, subLen));
36 |     if (list.length > 0) lists.push(list);
37 |   }
38 |
39 |   return ans.join(",");
40 | }
```

Python算法源码

```
1 # 输入获取
2 subLen = int(input())
3 n = int(input())
4 # 输入用例中的数组存在 4,,6 这种情况，所以这里需要过滤掉空串
5 lists = [list(filter(lambda x: x != "", input().split(","))) for _ in range(n)]
6
7
8 # 算法入口
9 def getResult():
10     ans = []
11
12     while len(lists) > 0:
13         lst = lists.pop(0)
14         tmp = lst[:subLen]
15         del lst[:subLen]
16         ans.extend(tmp)
17         if len(lst) > 0:
18             lists.append(lst)
19
20     return ",".join(ans)
21
22
23 # 算法调用
24 print(getResult())
```

C算法源码

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <stdbool.h>
```

运行

```
5 | 6 | #define MAX_N 1000
7 | #define MAX_LEN 101
8 |
9 | typedef struct List {
10 |     int size;
11 |     int nums[MAX_LEN];
12 | } List;
13 |
14 | void reverse(List *list) {
15 |     int l = 0;
16 |     int r = list->size - 1;
17 |
18 |     while (l < r) {
19 |         int tmp = list->nums[l];
20 |         list->nums[l] = list->nums[r];
21 |         list->nums[r] = tmp;
22 |         l++;
23 |         r--;
24 |     }
25 | }
26 |
27 | int main() {
28 |     // 每次读取的固定长度
29 |     int subLen;
30 |     scanf("%d", &subLen);
31 |
32 |     // 整数数组的数目
33 |     int n;
34 |     scanf("%d", &n);
35 |
36 |     List lists[n];
37 |     for (int i = 0; i < n; i++) {
38 |         lists[i].size = 0;
39 |     }
40 |
41 |     int total = 0;
42 |
```

```

43 |     for (int i = 0; i < n; i++) {
44 |         char s[10000];
45 |         scanf("%s", s);
46 |
47 |         char *token = strtok(s, ",");
48 |         while (token != NULL) {
49 |             if (strlen(token) > 0) {
50 |                 total++;
51 |                 lists[i].nums[lists[i].size++] = atoi(token);
52 |             }
53 |             token = strtok(NULL, ",");
54 |         }
55 |
56 |         reverse(&lists[i]);
57 |     }
58 |
59 |     int empty_size = 0;
60 |     bool empty[MAX_N] = {false};
61 |
62 |     int *res = (int *) malloc(sizeof(int) * total);
63 |     int res_size = 0;
64 |
65 |     while (empty_size < n) {
66 |         for (int i = 0; i < n; i++) {
67 |             for (int j = 0; j < subLen && lists[i].size > 0; j++) {
68 |                 res[res_size++] = lists[i].nums[--lists[i].size];
69 |             }
70 |
71 |             if (!empty[i] && lists[i].size == 0) {
72 |                 empty[i] = true;
73 |                 empty_size++;
74 |             }
75 |         }
76 |     }
77 |
78 |     for (int i = 0; i < res_size; i++) {
79 |         printf("%d", res[i]);

```



```
80 |
81 |         if (i < res_size - 1) {
82 |             printf(",");
83 |         }
84 |     }
85 |
86 |     return 0;
87 | }
```

C++ 算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  deque<int> splitCin(char delimiter) {
6      string s;
7      getline(cin, s);
8
9      stringstream ss(s);
10     string token;
11
12     deque<int> res;
13     while (getline(ss, token, delimiter)) {
14         if (!token.empty()) {
15             res.emplace_back(stoi(token));
16         }
17     }
18
19     return res;
20 }
21
22 int main() {
23     int subLen;
```

```
24     cin >> subLen;25
26     int n;
27     cin >> n;
28
29     cin.ignore();
30
31     deque<deque<int>> lists(n);
32     for (int i = 0; i < n; i++) {
33         lists.emplace_back(splitCin(','));
34     }
35
36     vector<int> res;
37
38     while (!lists.empty()) {
39         deque<int> list = lists.front();
40         lists.pop_front();
41
42         for (int i = 0; i < subLen && !list.empty(); i++) {
43             res.emplace_back(list.front());
44             list.pop_front();
45         }
46
47         if (!list.empty()) {
48             lists.push_back(list);
49         }
50     }
51
52     for (int i = 0; i < res.size(); i++) {
53         cout << res[i];
54
55         if (i < res.size() - 1) {
56             cout << ",";
57         }
58     }
59
60     return 0;
61 }
```

